

Motor de reglas de despacho

Documento de diseño · Matching, resolución de conflictos y trazabilidad

1. Requisitos

La solución debe permitir que una marca configure reglas de despacho y que un mismo pedido, evaluado bajo una misma versión de reglas, produzca siempre el mismo resultado.

R01	Determinismo: mismo pedido + misma versión de reglas = mismo resultado.
R02	Reglas: representar condiciones, acción, prioridad, versión y estado.
R03	Condiciones: monto, peso, comuna y tipo de entrega.
R04	Matching: identificar todas las reglas cuyas condiciones se cumplen.
R05	Acciones: modificar courier, precio o disponibilidad de tipo de entrega.
R06	Convivencia: reglas que modifican atributos diferentes pueden aplicarse simultáneamente.
R07	Conflictos: detectar reglas que modifican el mismo atributo con valores incompatibles.
R08	Prioridad: permitir que la marca determine qué regla prevalece ante un conflicto.
R09	Trazabilidad: registrar qué reglas participaron y por qué se obtuvo el resultado.
R10	Persistencia: conservar la decisión asociada a la versión de reglas utilizada.

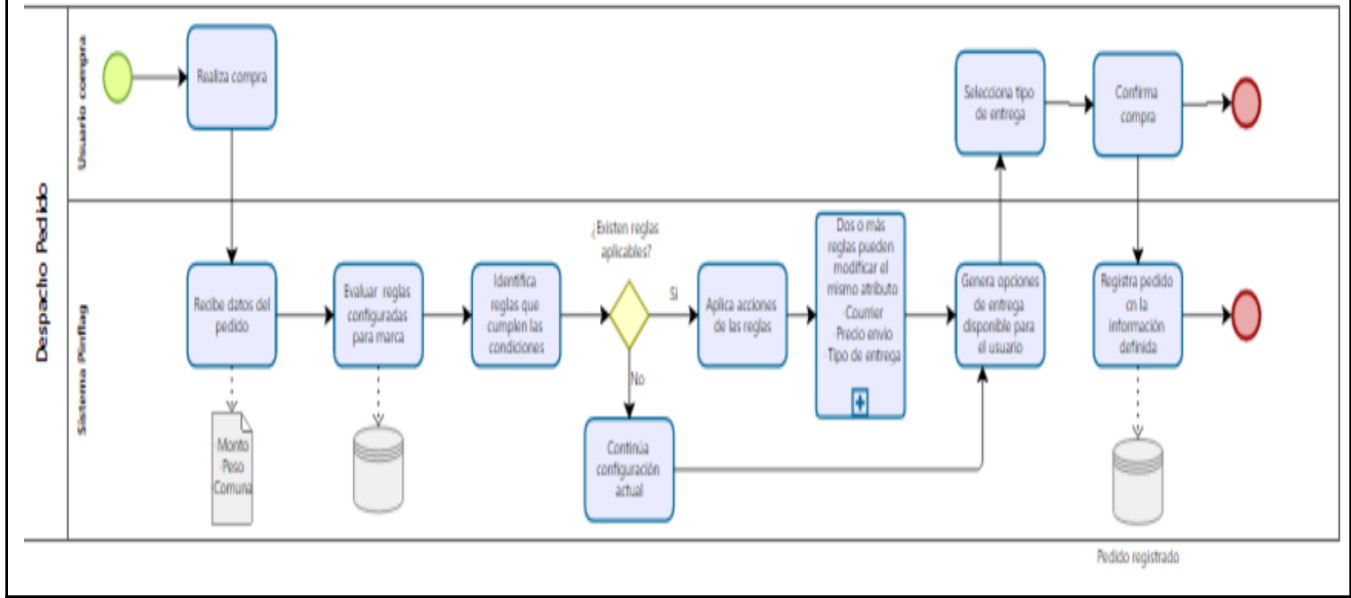
2. Situación actual (AS-IS)

El problema aparece cuando un pedido cumple múltiples reglas simultáneamente y no existe un mecanismo explícito para determinar cuál prevalece.

Pedido: \$60.000 / 2 kg Regla A → monto $\geq$ \$50.000 → courier = Blue Regla B → peso $<$ 20 kg → courier = FedEx

Ambas reglas coinciden y modifican el mismo atributo. Si la evaluación depende del orden de ejecución, el resultado puede cambiar y luego no hay una explicación clara de por qué se usó un courier determinado. Además, existen reglas incompatibles con ciertos tipos de entrega (p. ej., asignar un courier a un retiro en tienda).

DIAGRAMA AS-IS

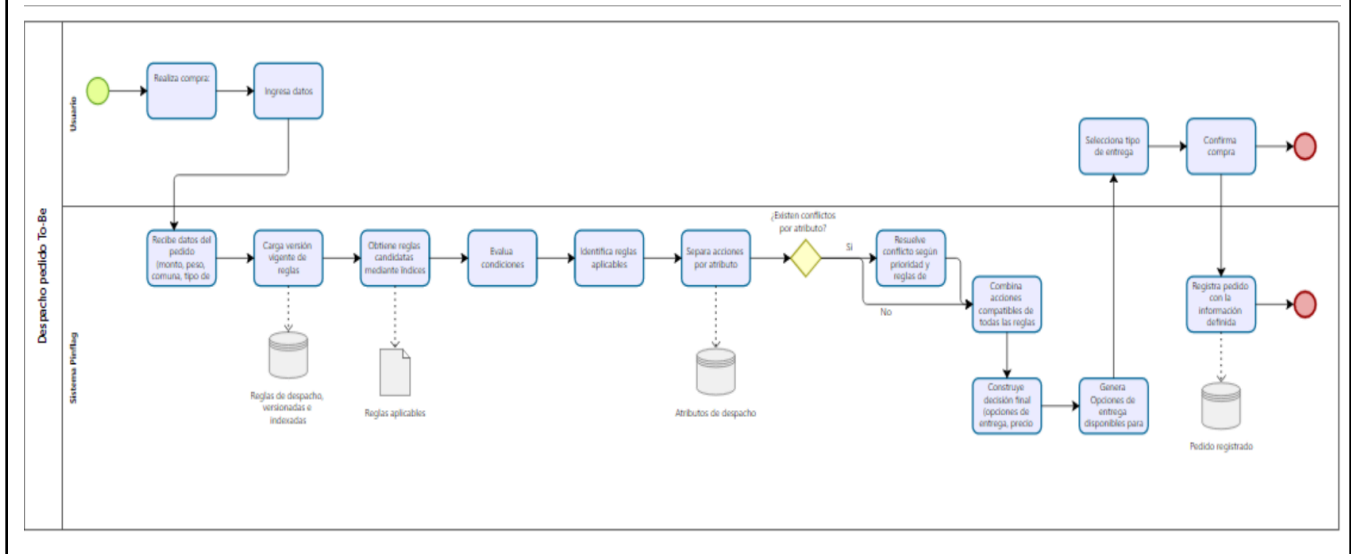


Motor de reglas de despacho · Página 1

3. Solución propuesta (TO-BE)

La solución separa el proceso en **matching**, **resolución de conflictos** y **trazabilidad**. Flujo: pedido → cargar versión de reglas → indexamiento → reglas candidatas → matching/evaluación → reglas aplicables → separar acciones por atributo → ¿conflicto? (**no**: combinar acciones compatibles; **sí**: resolver por prioridad) → decisión final → registrar trazabilidad → persistir decisión.

DIAGRAMA TO-BE



Arquitectura

Capas: **Frontend** (React + Vite) → **Application** (ServicioReglas) → **Domain** (MotorReglas: indexamiento, matching, conflictos, prioridad, decisión, trazabilidad) → **Infrastructure** (Express + PostgreSQL: reglas y decisiones).

Representación y evaluación

Una regla contiene condiciones, acción, prioridad, versión y estado. El motor usa un índice para reducir las reglas candidatas y luego evalúa las condiciones completas; el índice **no determina al ganador**.

- **Atributos distintos** coexisten: courier = Blue Express y precio = 0.
- **Mismo atributo, valores incompatibles** → conflicto; prevalece la regla de mayor prioridad (R1: Blue Express, prioridad 10 vs. R2: FedEx, prioridad 5 → **Blue Express**).
- Para asegurar determinismo, los conflictos incompatibles con **igual prioridad** se consideran inválidos en el prototipo.

Decisión y trazabilidad

La decisión se genera en el motor y se persiste con: versión de reglas, pedido evaluado, resultado, reglas evaluadas y aplicables, conflictos, reglas seleccionadas y motivo de selección. Esto permite reconstruir una decisión histórica sin volver a ejecutar el motor.

Reglas incompatibles

Las reglas incompatibles con un tipo de entrega se detectan antes de su utilización; el prototipo incorpora una validación simplificada que impide determinadas combinaciones inválidas.

Trade-offs y alcance

Para mantener el prototipo acotado: una condición y una acción por regla; indexamiento inicial simple; versionado básico; validación simplificada de compatibilidad; sin autenticación, autorización ni observabilidad productiva. La prioridad fue resolver de forma determinista **matching + conflictos + decisión + trazabilidad**, evitando sobre-diseñar el MVP.

4. Riesgos y mitigaciones

Crecimiento del número de reglas. Evaluar todas las reglas puede afectar el rendimiento.	Indexamiento para reducir candidatos; puede evolucionar a índices específicos por atributo y operador.
Reglas contradictorias o inválidas. Una marca puede configurar reglas que se pisen o no tengan sentido para un tipo de entrega.	Validación de reglas y detección de conflictos; la prioridad resuelve conflictos sin intervención de ingeniería.
Cambios posteriores en las reglas. Podría perderse la explicación de una decisión histórica.	Persistir la decisión junto con la versión de reglas y su trazabilidad.

Reglas existentes

Las reglas actualmente configuradas por las marcas deben migrarse al nuevo modelo y validarse antes de quedar activas. Las compatibles pueden conservarse; las incompatibles deben identificarse y corregirse antes de su publicación. En el prototipo, esta migración se representa mediante las reglas de ejemplo cargadas en PostgreSQL y la validación implementada en el motor.

5. Uso de IA

Se utilizó IA como apoyo durante el diseño y desarrollo: estructurar requisitos, explorar alternativas de modelamiento, identificar escenarios de conflicto y apoyar la implementación; también para revisar errores, proponer casos de prueba y apoyar la documentación.

A partir de las propuestas iniciales se tomaron decisiones según el alcance del problema, priorizando una solución simple basada en **indexamiento, matching, resolución explícita mediante prioridad y trazabilidad persistente**, y descartando complejidad que no aportaba al MVP. La implementación final fue revisada y ejecutada manualmente, probando escenarios de conflicto, reglas compatibles, reglas inválidas y pedidos sin coincidencias.